

Πανεπιστήμιο Πατρών
Τμήμα Μηχανικών Η/Υ και Πληροφορικής



Πρόβλεψη Χρήσης Πόρων στο Alibaba Cluster Set

Σύγκριση Μοντέλων Βαθιάς Μάθησης για Χρονοσειρές

Αναφορά Εργασίας

Γεωργία Μυλωνά, 1104783
Έλενα Βίτσιου, 1100502

May 19, 2026

Contents

1	Εισαγωγή	3
1.1	Github link	3
1.2	Σκοπός της εργασίας	3
1.3	Στόχοι της εργασίας	3
2	To Alibaba Cluster Dataset	5
2.1	Περιγραφή του Dataset	5
2.2	Ομαδοποίηση δεδομένων	5
3	Προεπεξεργασία Δεδομένων	6
3.1	Τα Στάδια της Προεπεξεργασίας	6
3.2	Διαχωρισμός των Δεδομένων	6
4	Ανάλυση Δεδομένων	8
4.1	Ανάλυση CPU και Memory Usage	8
4.1.1	Χαρακτηριστικά των Χρονοσειρών	8
4.1.2	Στατιστικά Χαρακτηριστικά	8
4.2	Αποσύνθεση της χρονοσειράς	8
4.2.1	Σκοπός και Μέθοδος	8
4.2.2	Αποτελέσματα του decomposition	9
4.3	Ανάλυση της αυτοσυσχέτισης	9
4.3.1	Θεωρητικό Υπόβαθρο	9
4.3.2	Ευρήματα	9
5	Αρχιτεκτονικές Μοντέλων	10
5.1	Long Short-Term Memory (LSTM)	10
5.1.1	Θεωρία και Αρχιτεκτονική	10
5.1.2	Δομή του LSTM Μοντέλου	10
5.1.3	Αποτελέσματα LSTM	11
5.2	Gated Recurrent Unit (GRU)	11
5.2.1	Θεωρία και Αρχιτεκτονική	11
5.2.2	Δομή του GRU Μοντέλου	12
5.2.3	Αποτελέσματα GRU	12
5.3	Transformer	12
5.3.1	Θεωρία και Αρχιτεκτονική	12
5.3.2	Δομή του Transformer Μοντέλου και Προσαρμογές	13
5.3.3	Αποτελέσματα Transformers και Δυσκολίες Εκπαίδευσης	13
5.3.4	Ανάλυση της Απόδοσης και Αιτίες Αποτυχίας	14

6 Σύγκριση Αποτελεσμάτων	15
6.1 Συγκριτικός Πίνακας	15
6.2 Ανάλυση των μετρικών	15
6.2.1 Mean Squared Error (MSE)	15
6.2.2 Mean Absolute Error (MAE)	15
6.2.3 Mean Absolute Percentage Error (MAPE)	15
6.3 Ανάλυση ανά Horizon	16
6.3.1 Short Horizon Performance	16
6.3.2 Long Horizon Performance	16
7 Inference time	17
7.1 Τα αποτελέσματα του inference time	17
7.2 Ανάλυση	17
8 Συμπεράσματα	19
8.1 Κύρια Ευρήματα	19
8.2 Σημασία Αποτελεσμάτων	19
8.3 Προτάσεις για Μελλοντικές Εργασίες	19
9 Βιβλιογραφία	20

Chapter 1

Εισαγωγή

1.1 Github link

<https://github.com/grgmyl/alibabacuster.git>

1.2 Σκοπός της εργασίας

Η παρούσα εργασία επικεντρώνεται στην πρόβλεψη της χρήσης υπολογιστικών πόρων σε bare metal nodes, αξιοποιώντας δεδομένα χρονοσειρών από CPU usage και memory usage. Αυτό το πρόβλημα ενέχει σημαντική πρακτική σημασία, καθώς η ακριβής πρόβλεψη της χρήσης πόρων είναι ζωτικής σημασίας για την αποτελεσματική διαχείριση και την κατανομή πόρων σε σύγχρονα cloud περιβάλλοντα. Στόχος της εργασίας είναι η ανάπτυξη και αξιολόγηση μοντέλων βαθιάς μάθησης που μπορούν να προβλέψουν με ακρίβεια τη μελλοντική χρήση πόρων. Τα μοντέλα που αναπτύχθηκαν καλύπτουν διάφορες αρχιτεκτονικές νευρωνικών δικτύων, συμπεριλαμβανομένων των Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU) και Transformer-based μοντέλων. Η μελέτη αποσκοπεί στην κατανόηση της κατάλληλης επιλογής αρχιτεκτονικής για τέτοια προβλήματα και στην εξαγωγή συμπερασμάτων σχετικά με την αποτελεσματικότητα διαφόρων προσεγγίσεων βαθιάς μάθησης στο πεδίο της πρόβλεψης χρονοσειρών.

1.3 Στόχοι της εργασίας

Οι βασικοί στόχοι της εργασίας είναι:

- Ανάλυση και προεπεξεργασία των δεδομένων CPU και memory usage από το Alibaba Cluster dataset
- Ανάπτυξη ρεαλιστικών χρονοσειρών συγκεντρώνοντας δεδομένα από πολλαπλά machines
- Εκπαίδευση και αξιολόγηση τριών κύριων αρχιτεκτονικών νευρωνικών δικτύων: LSTM, GRU και Transformers
- Πρόβλεψη τόσο για short horizon (5 timesteps) όσο και για long horizon (10 timesteps)
- Αξιολόγηση των μοντέλων με τις μετρικές Mean Squared Error (MSE), Mean Absolute Error (MAE) και Mean Absolute Percentage Error (MAPE)

- Σύγκριση των μοντέλων ως προς την ακρίβεια, την ταχύτητα inference και την υπολογιστική πολυπλοκότητα

Chapter 2

To Alibaba Cluster Dataset

2.1 Περιγραφή του Dataset

Το dataset που χρησιμοποιήθηκε προέρχεται από το Alibaba Cluster Set, ένα δημόσια διαθέσιμο σύνολο δεδομένων που συγκεντρώνει πραγματικές μετρήσεις χρήσης πόρων από ένα μεγάλο cloud cluster. Αυτό το dataset παρέχει μετρήσεις CPU και memory usage από χιλιάδες φυσικές μηχανές (machines) σε πραγματικό περιβάλλον παραγωγής. Το σύνολο δεδομένων περιείχε συνολικά 10.283 διαφορετικά machine identifiers (IDs), με κάθε μηχανή να έχει τις δικές της χρονοσειρές μετρήσεων. Οι τιμές καταγράφονταν σε τακτά χρονικά διαστήματα, παρέχοντας μια λεπτομερή άποψη της δυναμικής χρήσης πόρων.

2.2 Ομαδοποίηση δεδομένων

- Ομαδοποίηση όλων των καταγραφών ανά timestamp, ανεξάρτητα από τη μηχανή προέλευσης
- Υπολογισμό της μέσης τιμής CPU usage για όλες τις μηχανές σε κάθε timestamp
- Δημιουργία μιας ενιαίας χρονοσειράς που αντιπροσωπεύει το συνολικό pattern χρήσης πόρων στο cluster

Chapter 3

Προεπεξεργασία Δεδομένων

3.1 Τα Στάδια της Προεπεξεργασίας

Η προεπεξεργασία των δεδομένων αποτέλεσε ένα κρίσιμο στάδιο της εργασίας, καθώς η ποιότητα και η σωστή μορφοποίηση των δεδομένων εισόδου επηρεάζουν άμεσα την αποτελεσματικότητα των μοντέλων μάθησης. Τα στάδια που εφαρμόστηκαν ήταν τα ακόλουθα:

Αφαίρεση διπλότυπων εγγραφών: Πρώτα, εντοπίστηκαν και αφαιρέθηκαν οποιεσδήποτε διπλότυπες εγγραφές που θα μπορούσαν να εισάγουν bias στα δεδομένα εκπαίδευσης.

Διαχείριση missing values: Για τις χρονοσειρές, η ύπαρξη missing values είναι συχνό φαινόμενο. Εφαρμόστηκε linear interpolation για την εκτίμηση των ελλειπών τιμών, δεδομένου ότι αυτή η μέθοδος είναι κατάλληλη για χρονοσειρές που παρουσιάζουν σχετική ομαλότητα.

Αφαίρεση outliers: Οι ακραίες τιμές που ενδέχεται να προκύπτουν από σφάλματα μέτρησης ή ανώμαλα γεγονότα αφαιρέθηκαν χρησιμοποιώντας την μέθοδο του Interquartile Range (IQR).

Κανονικοποίηση δεδομένων: Αναγκαία ήταν η κανονικοποίηση των δεδομένων στο εύρος $[0, 1]$ χρησιμοποιώντας τον MinMaxScaler. Αυτό επιτρέπει στα νευρωνικά δίκτυα να εκπαιδεύονται πιο αποτελεσματικά και αποτρέπει τα προβλήματα που σχετίζονται με τις διαφορετικές κλίμακες των χαρακτηριστικών.

Δημιουργία sequences: Τα νευρωνικά δίκτυα που χρησιμοποιούνται για χρονοσειρές απαιτούν δεδομένα σε μορφή ακολουθιών. Για τα μοντέλα GRU και LSTM δημιουργήθηκαν sequences μεγέθους 24, ενώ για τον Transformer χρησιμοποιήθηκαν sequences μεγέθους 36, προκειμένου να προβλεφθούν τα επόμενα 5 ή 10 timesteps, ανάλογα με τον χρονικό ορίζοντα.

3.2 Διαχωρισμός των Δεδομένων

Τα δεδομένα χωρίστηκαν σε τρία διαφορετικά σύνολα για την ορθή αξιολόγηση των μοντέλων:

- **Training set (80%):** Χρησιμοποιήθηκε για την εκπαίδευση των μοντέλων, δηλαδή για τη βελτιστοποίηση των παραμέτρων τους.
- **Validation set (10%):** Χρησιμοποιήθηκε κατά την εκπαίδευση για τη ρύθμιση των hyperparameters και την πρόληψη της υπερεκπαίδευσης μέσω του Early Stopping.

- **Test set (10%):** Χρησιμοποιήθηκε για την τελική αξιολόγηση της απόδοσης του μοντέλου σε δεδομένα που δεν έχουν δει πριν.

Αυτό το 80-10-10 split είναι μια κλασική διαίρεση που ισορροπεί ανάμεσα στην αρκετή εκπαίδευση και την αξιόπιστη αξιολόγηση.

Chapter 4

Ανάλυση Δεδομένων

4.1 Ανάλυση CPU και Memory Usage

4.1.1 Χαρακτηριστικά των Χρονοσειρών

Η ανάλυση των δεδομένων αποκάλυψε σημαντικές διαφορές μεταξύ των CPU και memory usage patterns. Η CPU usage παρουσίασε μεγαλύτερη μεταβλητότητα χρόνο προς χρόνο, με σαφείς περίοδοι αιχμής και χαμηλότερης χρήσης. Αυτό είναι λογικό, καθώς η CPU usage είναι πιο δυναμική και αντιδρά άμεσα στις εντολές που εκτελούνται. Αντίθετα, η memory usage εμφάνισε πιο ομαλή και σταθερή συμπεριφορά, χαμηλότερη μεταβλητότητα και γενικά σαφώς λιγότερη προβλεψιμότητα. Η memory χρήση τείνει να είναι πιο αθροιστική και να αλλάζει λιγότερο συχνά, καθώς τα δεδομένα παραμένουν στη μνήμη μεταξύ διαδοχικών χρονικών περιόδων. Λαμβάνοντας υπόψη τις παραπάνω παρατηρήσεις και το γεγονός ότι η CPU usage παρουσίασε σαφώς καλύτερα χαρακτηριστικά προβλεψιμότητας, αποφασίστηκε να επικεντρωθεί η πρόβλεψη αποκλειστικά στο CPU usage. Αυτή η απόφαση επέτρεψε βαθύτερη ανάλυση και πιο στοχευμένη βελτιστοποίηση των μοντέλων.

4.1.2 Στατιστικά Χαρακτηριστικά

Οι βασικές στατιστικές του CPU usage κατέδειξαν:

- Μέση τιμή: $\mu \approx 0.62$ (κανονικοποιημένη κλίμακα)
- Τυπική απόκλιση: $\sigma \approx 0.062$
- Ελάχιστη τιμή: $x_{\min} \approx 0.48$
- Μέγιστη τιμή: $x_{\max} \approx 0.69$

4.2 Αποσύνθεση της χρονοσειράς

4.2.1 Σκοπός και Μέθοδος

Η αποσύνθεση μιας χρονοσειράς είναι μια σημαντική τεχνική για την κατανόηση των στοιχείων που συνθέτουν μια παρατηρούμενη χρονοσειρά. Εφαρμόστηκε η κλασική προσθετική decomposition (additive decomposition), που διαχωρίζει τη χρονοσειρά σε τέσσερα κύρια συστατικά:

$$X_t = T_t + S_t + R_t \quad (4.1)$$

όπου T_t είναι η τάση (trend), S_t είναι η εποχικότητα (seasonal component), και R_t είναι τα κατάλοιπα (residuals).

4.2.2 Αποτελέσματα του decomposition

Τα αποτελέσματα της αποσύνθεσης έδειξαν:

Trend Component: Η CPU usage παρουσίασε ένα ισχυρό upward trend κατά την πρώτη περίοδο, ακολουθούμενο από μια σχετική σταθεροποίηση. Αυτό υποδηλώνει αυξανόμενη φόρτιση του cluster ακολουθούμενη από μια περίοδο σταθερής χρήσης.

Seasonal Component: Παρατηρήθηκε σαφής εποχικότητα με περιοδικότητα που μπορεί να συσχετιστεί με τα ωρήσια ή ημερήσια patterns χρήσης του cluster. Η amplitude της εποχικότητας ήταν σημαντική, υποδηλώνοντας συνεπή patterns στην χρήση πόρων.

Residual Component: Τα κατάλοιπα παρουσίασαν ένα ποσοστό τυχαιότητας, αλλά η δομή τους δεν ήταν πλήρως τυχαία, υποδηλώνοντας την ύπαρξη περαιτέρω δομών που δεν εξηγούνται από τα trend και seasonal components.

4.3 Ανάλυση της αυτοσυσχέτισης

4.3.1 Θεωρητικό Υπόβαθρο

Η ανάλυση αυτοσυσχέτισης (autocorrelation) είναι απαραίτητη για την κατανόηση της χρονικής εξάρτησης σε μια χρονοσειρά. Η συνάρτηση αυτοσυσχέτισης (ACF) μετρά τη συσχέτιση μεταξύ των τιμών μιας χρονοσειράς σε διαφορετικές χρονικές υστερήσεις (lags):

$$\rho(h) = \frac{\sum_{t=1}^{n-h} (x_t - \bar{x})(x_{t+h} - \bar{x})}{\sum_{t=1}^n (x_t - \bar{x})^2} \quad (4.2)$$

όπου h είναι το lag, n είναι το μήκος της χρονοσειράς, και $\bar{x}$ είναι ο μέσος όρος.

4.3.2 Ευρήματα

Η ACF της CPU usage έδειξε ισχυρή θετική αυτοσυσχέτιση για ένα μεγάλο αριθμό lags. Συγκεκριμένα:

- Η συσχέτιση παρέμεινε υψηλή (> 0.5) για τουλάχιστον τις πρώτες 20 χρονικές υστερήσεις
- Παρατηρήθηκαν περιοδικές κορυφές στη συσχέτιση, συνεπές με την εποχικότητα που παρατηρήθηκε στην decomposition
- Αυτό υποδηλώνει ισχυρή χρονική μνήμη: η παρούσα τιμή είναι σημαντικά εξαρτώμενη από τις προηγούμενες τιμές

Αυτή η ισχυρή αυτοσυσχέτιση είναι θετική για τη χρήση των RNN-based μοντέλων (LSTM, GRU), καθώς αυτά τα μοντέλα είναι ιδιαίτερα κατάλληλα για ακολουθίες με χρονική εξάρτηση.

Chapter 5

Αρχιτεκτονικές Μοντέλων

5.1 Long Short-Term Memory (LSTM)

5.1.1 Θεωρία και Αρχιτεκτονική

Τα LSTM δίκτυα είναι ένας τύπος Recurrent Neural Network (RNN) που σχεδιάστηκαν για να αντιμετωπίσουν το πρόβλημα της εξαφάνισης κλίσης (vanishing gradient problem) που αντιμετωπίζουν τα παραδοσιακά RNNs. Η βασική δομή ενός LSTM κελιού περιλαμβάνει:

- **Forget Gate:** Ελέγχει ποια πληροφορία από το προηγούμενο κελί θα πρέπει να "ξεχαστεί"
- **Input Gate:** Καθορίζει ποιες νέες πληροφορίες θα προστεθούν στη κατάσταση του κελιού
- **Cell State:** Διατηρεί τη μνήμη του δικτύου σε μεγάλες χρονικές αποστάσεις
- **Output Gate:** Καθορίζει ποια μέρος της κατάστασης του κελιού θα εξοδοποιηθεί

Μαθηματικά, ένα LSTM κελί περιγράφεται από τις εξής εξισώσεις:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (5.1)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (5.2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (5.3)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (5.4)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (5.5)$$

$$h_t = o_t * \tanh(C_t) \quad (5.6)$$

όπου σ είναι η logistic sigmoid συνάρτηση και τα W , b είναι τα βάρη και τα biases του δικτύου.

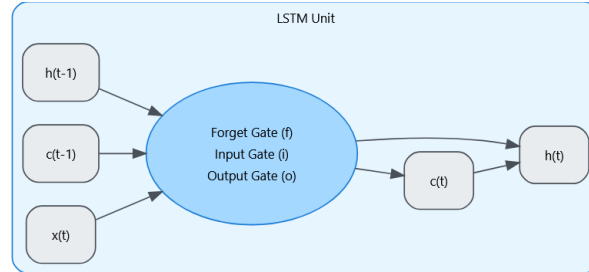
5.1.2 Δομή του LSTM Μοντέλου

Το LSTM μοντέλο υλοποιήθηκε σε PyTorch και αποτελείται από:

- Δύο στοιβαγμένα LSTM layers με **64 μονάδες** (hidden size) το καθένα (num_layers=2)

- Dropout με ρυθμό 0.2 μεταξύ των layers για πρόληψη της υπερεκπαίδευσης
- Ένα Dense (fully connected) output layer που παράγει *horizon* τιμές εξόδου

Η είσοδος στο μοντέλο αποτελείται από ακολουθίες μεγέθους 24 timesteps (LOOKBACK = 24), και η εκπαίδευση πραγματοποιήθηκε με τον βελτιστοποιητή Adam, χρησιμοποιώντας MSE ως συνάρτηση κόστους και Early Stopping με patience = 10.



Αρχιτεκτονική LSTM μοντέλου

5.1.3 Αποτελέσματα LSTM

Το μοντέλο LSTM παρουσίασε σταθερή και αξιόπιστη απόδοση:

- **Short Horizon:** MSE = 0.000232, MAE = 0.013694, MAPE = 2.77%
- **Long Horizon:** MSE = 0.000269, MAE = 0.014183, MAPE = 2.91%

Ενδιαφέρον είναι ότι το LSTM παρουσίασε καλύτερη απόδοση στο short horizon σε σύγκριση με το long horizon ως προς το MSE. Αυτό μπορεί να ερμηνευθεί ως ότι το LSTM χρειάζεται περισσότερο "context" από τα δεδομένα για να λειτουργήσει αποτελεσματικά, κάτι που εξηγεί τη σχετικά χαμηλότερη απόδοσή του στο long horizon.

5.2 Gated Recurrent Unit (GRU)

5.2.1 Θεωρία και Αρχιτεκτονική

Τα GRU δίκτυα είναι μια απλούστερη εκδοχή των LSTM δικτύων, προτάθηκαν ως ένα πιο υπολογιστικά αποδοτικό εναλλακτικό. Ενώ διατηρούν τη δυνατότητα να ακολουθούν μεγάλες χρονικές εξαρτήσεις, έχουν λιγότερες πύλες:

- **Reset Gate:** Ελέγχει πόσα από τα προηγούμενα κρυμμένα δεδομένα θα πρέπει να "ξεχαστούν"
- **Update Gate:** Αποφασίζει πόσα ενημερώσιμα υπάρχουν για τη κρυμμένη κατάσταση

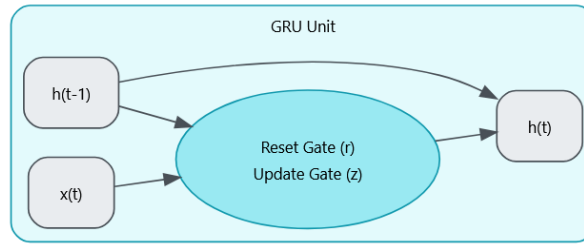
Ένα GRU κελί περιγράφεται από τις εξής εξισώσεις:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (5.7)$$

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (5.8)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t] + b) \quad (5.9)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (5.10)$$



Αρχιτεκτονική GRU μοντέλου

5.2.2 Δομή του GRU Μοντέλου

Το GRU μοντέλο υλοποιήθηκε σε Keras/TensorFlow και αποτελείται από:

- Ένα GRU layer με **64 μονάδες** (return_sequences=True)
- Dropout layer με ρυθμό 0.2
- Δεύτερο GRU layer με **64 μονάδες**
- Dropout layer με ρυθμό 0.2
- Ένα Dense output layer
- **Short Horizon:** MSE = 0.000147, MAE = 0.009651, MAPE = 1.59%
- **Long Horizon:** MSE = 0.000314, MAE = 0.015301, MAPE = 2.52%

5.2.3 Αποτελέσματα GRU

Το μοντέλο GRU παρουσίασε την καλύτερη συνολική απόδοση μεταξύ όλων των μοντέλων που δοκιμάστηκαν:

Αυτά είναι τα καλύτερα αποτελέσματα σε όλες τις μετρικές, ιδιαίτερα στο short horizon. Η απόδοση του GRU υποδηλώνει ότι, αν και απλούστερο από το LSTM, το GRU είναι αρκετά εκφραστικό για να κατανοήσει τα patterns σε αυτό το σύνολο δεδομένων.

5.3 Transformer

5.3.1 Θεωρία και Αρχιτεκτονική

Τα Transformers αποτελούν μια σχετικά νέα αρχιτεκτονική που βασίζεται εξολοκλήρου στο μηχανισμό της προσοχής (attention mechanism). Αντίθετα με τα RNNs, τα Transformers δεν εξαρτώνται από επαναληπτικές συνδέσεις, γεγονός που τους επιτρέπει να επεξεργάζονται ολόκληρες ακολουθίες παράλληλα.

Ο κύριος μηχανισμός είναι ο scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5.11)$$

όπου Q είναι τα queries, K είναι τα keys, V είναι τα values, και d_k είναι η διάσταση των keys.

5.3.2 Δομή του Transformer Μοντέλου και Προσαρμογές

Το Transformer μοντέλο υλοποιήθηκε σε Keras/TensorFlow και αποτελείται από:

- Ένα Conv1D layer (filters=32, kernel_size=3) για τοπική εξαγωγή χαρακτηριστικών από την είσοδο
- Ένα **Positional Embedding layer** (learned embeddings μέσω Embedding layer με d_model=32) για την κωδικοποίηση της χρονικής θέσης κάθε timestep
- Δύο Transformer Encoder blocks, ο καθένας με:
 - Multi-Head Attention (2 heads, key_dim=16, dropout=0.1)
 - Feed-Forward Network (FFN) με FF_DIM=64 και relu ενεργοποίηση
 - Residual connections και Layer Normalization
- Ένα Dense layer (32 μονάδες, relu) ακολουθούμενο από Dropout
- Ένα Dense output layer που παράγει *horizon* τιμές εξόδου

Η εκπαίδευση του Transformer αποδείχθηκε ιδιαίτερα απαιτητική και απαίτησε σημαντικές προσαρμογές και ρυθμίσεις των hyperparameters. Λόγω της εγγενούς πολυπλοκότητας της αρχιτεκτονικής και της δυσκολίας της να καταλάβει τα time series patterns, αναγκαστήκαμε να κάνουμε τις εξής αλλαγές:

Πρώτον, **αυξήσαμε σημαντικά τον αριθμό των epochs** κατά την εκπαίδευση σε 80 epochs (έναντι 40 για LSTM και GRU). Ενώ τα LSTM και GRU συγκλίνουν σε ικανοποιητικές λύσεις σχετικά γρήγορα, το Transformer απαίτησε πολλαπλάσιο αριθμό επαναλήψεων για να αρχίσει να μαθαίνει τα underlying patterns στα time series δεδομένα.

Δεύτερον, **τροποποιήσαμε το μήκος της ακολουθίας** σε 36 timesteps (έναντι 24 για GRU/LSTM). Αυτή η αύξηση οδήγησε σε εκθετική αύξηση της υπολογιστικής πολυπλοκότητας λόγω της φύσης του attention mechanism, αλλά ήταν απαραίτητη για την αναγνώριση μακροχρόνιων εξαρτήσεων.

Τρίτον, **χρησιμοποιήσαμε learned positional embeddings** αντί για sinusoidal encodings. Το positional embedding layer εκπαιδεύεται από κοινού με το υπόλοιπο μοντέλο, επιτρέποντάς του να μάθει χρήσιμες αναπαραστάσεις θέσης για τα συγκεκριμένα δεδομένα.

Τέταρτον, **χρησιμοποιήσαμε Huber loss** αντί για MSE ως συνάρτηση κόστους, καθώς προσφέρει μεγαλύτερη ανθεκτικότητα σε outliers. Παράλληλα, εφαρμόστηκε ReduceLROnPlateau callback για αυτόματη μείωση του learning rate (αρχική τιμή 10^{-4} , με factor 0.5 και patience=5). Το Early Stopping για τον Transformer χρησιμοποιεί patience = 12 (έναντι 10 για GRU/LSTM), δίνοντας στο μοντέλο περισσότερες ευκαιρίες βελτίωσης λόγω της αργότερης σύγκλισής του.

Πέμπτον, **προστέθηκε ένα Conv1D layer** πριν τα transformer blocks για τοπική εξαγωγή χαρακτηριστικών, βοηθώντας το μοντέλο να εντοπίσει τοπικά patterns στη χρονοσειρά πριν από την εφαρμογή του global attention mechanism.

5.3.3 Αποτελέσματα Transformers και Δυσκολίες Εκπαίδευσης

Αντίθετα με τις αναμενόμενες προσδοκίες, τα Transformers δεν παρουσίασαν τα καλύτερα αποτελέσματα:

- **Short Horizon:** MSE = 0.000518, MAE = 0.018911, MAPE = 3.14%

- **Long Horizon:** MSE = 0.000961, MAE = 0.026394, MAPE = 4.40%

Τα αποτελέσματα δείχνουν σημαντικά αυξημένα σφάλματα, ιδιαίτερα στο long horizon.

5.3.4 Ανάλυση της Απόδοσης και Αιτίες Αποτυχίας

Παρά τις σημαντικές προσαρμογές που κάναμε στα hyperparameters, οι Transformers συνέχισαν να υστερούν. Υπάρχουν πολλαπλοί παράγοντες που συνέβαλαν σε αυτή την αποτυχία:

Πρώτον, η **πολυπλοκότητα του μοντέλου** έναντι του **μεγέθους του dataset** δημιουργεί ένα μη ευνοϊκό ισοζύγιο. Οι Transformers έχουν ένα τεράστιο αριθμό παραμέτρων που πρέπει να μάθουν, αλλά τα δεδομένα χρήσης πόρων, αν και aggregated, δεν αρκούν για να καθοδηγήσουν αποτελεσματικά τη διαδικασία εκπαίδευσης σε ένα τόσο μεγάλο χώρο παραμέτρων.

Δεύτερον, τα **time series δεδομένα χρήσης πόρων έχουν ισχυρή χρονική δομή** που είναι φυσικό για τα RNNs αλλά όχι για τους Transformers. Ενώ τα learned positional embeddings βοηθούν το μοντέλο να κατανοήσει τη χρονική σειρά, αυτή η πληροφορία δεν ενσωματώνεται τόσο φυσικά όσο στις αναδρομικές δομές των RNNs.

Τρίτον, οι αυξήσεις στον αριθμό των epochs, στο sequence length (36), και στο batch size δημιουργούν **αύξηση της υπολογιστικής πολυπλοκότητας** που δεν αποδίδει καλύτερα αποτελέσματα. Ο Transformer μαθαίνει πιο αργά και λιγότερο αποτελεσματικά από τα RNNs για αυτόν τον τύπο δεδομένων.

Τέλος, υπάρχει το **θεμελιώδες ζήτημα της σχεδίασης**: οι Transformers σχεδιάστηκαν για εφαρμογές όπου η δυνατότητα να κατανοηθούν σχέσεις μεταξύ απόμακρων στοιχείων είναι κρίσιμη. Αλλά σε χρονοσειρές όπου τα patterns είναι κυρίως sequential και τοπικά, τα RNNs με τη διαδοχική τους επεξεργασία είναι πολύ πιο κατάλληλα.

Chapter 6

Σύγκριση Αποτελεσμάτων

6.1 Συγκριτικός Πίνακας

Table 6.1: Σύγκριση απόδοσης των μοντέλων σε αμφότερα τα ορίζοντα πρόβλεψης

Μοντέλο	Horizon	MSE	MAE	MAPE
LSTM	Short	0.000232	0.013694	2.77%
LSTM	Long	0.000269	0.014183	2.91%
GRU	Short	0.000147	0.009651	1.59%
GRU	Long	0.000314	0.015301	2.52%
Transformer	Short	0.000518	0.018911	3.14%
Transformer	Long	0.000961	0.026394	4.40%

6.2 Ανάλυση των μετρικών

6.2.1 Mean Squared Error (MSE)

Το MSE είναι μια μετρική που δίνει περισσότερο βάρος στα μεγάλα σφάλματα. Στο short horizon, το GRU αποδίδει περίπου 72% καλύτερα από το Transformer (MSE 0.000147 vs 0.000518) και περίπου 37% καλύτερα από το LSTM (MSE 0.000147 vs 0.000232). Στο long horizon, το LSTM έχει την καλύτερη απόδοση με MSE = 0.000269.

6.2.2 Mean Absolute Error (MAE)

Το MAE παρέχει μια πιο ευθεία μετρική του μέσου σφάλματος. Το GRU έχει συνεχώς τα κατώτερα σφάλματα, με MAE = 0.009651 στο short horizon (περίπου 30% καλύτερα από το LSTM) και 7.7% καλύτερα από το LSTM στο long horizon.

6.2.3 Mean Absolute Percentage Error (MAPE)

Το MAPE είναι ιδιαίτερα χρήσιμο καθώς παρέχει ένα ποσοστό του σφάλματος σε σχέση με τις πραγματικές τιμές. Το GRU παρουσιάζει πάλι την καλύτερη απόδοση με 1.59% στο short horizon και 2.52% στο long horizon.

6.3 Ανάλυση ανά Horizon

6.3.1 Short Horizon Performance

Στη short-term πρόβλεψη, όπου πρόκειται να προβλέψουμε τα επόμενα 5 timesteps, τα μοντέλα αποδίδουν σχετικά καλά. Το GRU είναι ευδιάκριτα ανώτερο, ενώ το LSTM αποδίδει ικανοποιητικά και ο Transformer αποδίδει αρκετά χειρότερα.

6.3.2 Long Horizon Performance

Στη long-term πρόβλεψη, όπου πρόκειται να προβλέψουμε τα επόμενα 10 timesteps, η απόδοση όλων των μοντέλων χειροτερεύει, πράγμα που είναι αναμενόμενο. Ωστόσο, η υποβάθμιση στην απόδοση δεν είναι ομοιόμορφη:

- Το LSTM παρουσιάζει ελαφρώς καλύτερη απόδοση στο long horizon ως προς το MSE
- Το GRU παρουσιάζει σχετικά σταθερή απόδοση
- Ο Transformer παρουσιάζει σημαντική υποβάθμιση

Chapter 7

Inference time

7.1 Τα αποτελέσματα του inference time

Τα αποτελέσματα έδειξαν ξεκάθαρες διαφορές μεταξύ των μοντέλων στη ταχύτητα εκτέλεσης των προβλέψεων:

Inference time ανά μοντέλο και horizon

Model	Horizon	Inference time(s)
LSTM	Short	0.001010
LSTM	Long	0.000942
GRU	Short	0.016376
GRU	Long	0.025077
Transformers	Short	1.981681
Transformers	Long	2.468906

7.2 Ανάλυση

Το inference time μετρά τον χρόνο που απαιτείται για την πραγματοποίηση predictions στο test set και αποτελεί ένα κρίσιμο κριτήριο αξιολόγησης για εφαρμογές real-time resource prediction. Τα αποτελέσματα αποκαλύπτουν σημαντικές διαφορές στην υπολογιστική απόδοση των διαφόρων αρχιτεκτονικών.

Το LSTM παρουσίασε πολύ χαμηλό inference time και στα δύο horizons, διατηρώντας παράλληλα σταθερή απόδοση και ικανοποιητική ακρίβεια πρόβλεψης. Αξίζει να σημειωθεί ότι το LSTM υλοποιήθηκε σε PyTorch με χρήση GPU (εφόσον διαθέσιμο), γεγονός που συμβάλλει στο εξαιρετικά χαμηλό inference time. Η δομή των στοιβαγμένων LSTM layers με 64 μονάδες επιτρέπει αποδοτική επεξεργασία χωρίς περιττή υπολογιστική επιβάρυνση.

Το GRU έπεται ως προς ακρίβεια, αλλά με αισθητά μεγαλύτερο inference time. Ειδικά στον short horizon, το GRU χρειάζεται περίπου 0.016s ανά πρόβλεψη, ενώ στον long horizon αυτός ο χρόνος αυξάνεται σε 0.025s — περίπου 26x περισσότερο από το LSTM. Αυτή η διαφορά αποδίδεται εν μέρει στο γεγονός ότι το GRU υλοποιήθηκε σε Keras/TensorFlow, ενώ το LSTM σε PyTorch.

Οι Transformers είχαν εμφανώς μεγαλύτερο inference time, με τιμές που κυμαίνονται από 1.98s έως 2.47s. Αυτό είναι ακόμα και 1980x πιο αργό από το LSTM — μια διαφορά που δεν μπορεί να αγνοηθεί σε εφαρμογές παραγωγής. Η τεράστια αυτή υπολογιστική επιβάρυνση αποδίδεται άμεσα στο attention mechanism, το οποίο απαιτεί τετραγωνική πολυπλοκότητα $O(n^2)$ ως προς το μήκος της

ακολουθίας n . Η χρήση μεγαλύτερου sequence length (36 έναντι 24) επιδεινώνει περαιτέρω αυτό το πρόβλημα.

Αυτά τα αποτελέσματα έχουν σημαντικές επιπτώσεις για την επιλογή του κατάλληλου μοντέλου σε σενάρια παραγωγής. Ενώ οι Transformers παρουσίασαν δυσκολίες στην ακρίβεια, ο εξαιρετικά υψηλός χρόνος inference τα καθιστά πρακτικά ακατάλληλους για εφαρμογές που απαιτούν γρήγορη απόκριση. Αντίθετα, το LSTM και το GRU εμφανίζουν αποδεκτούς χρόνους εκτέλεσης που είναι συμβατοί με real-time προβλέψεις. Το συμπέρασμα είναι ότι, για χρονοσειρές με ξεκάθαρη δομή και σχετικά μικρά ή μεσαία μήκη ακολουθιών, τα παραδοσιακά RNN-based μοντέλα παραμένουν η προτιμητέα επιλογή.

Chapter 8

Συμπεράσματα

8.1 Κύρια Ευρήματα

Τα αποτελέσματα της εργασίας αποδεικνύουν ότι οι αρχιτεκτονικές RNN, ιδιαίτερα τα GRU, παραμένουν ιδιαίτερα αποτελεσματικές για την πρόβλεψη χρονοσειρών χρήσης πόρων σε cloud περιβάλλοντα. Συγκεκριμένα:

- Το μοντέλο GRU παρουσίασε την καλύτερη συνολική απόδοση με MAPE = 1.59% στο short horizon και 2.52% στο long horizon
- Το LSTM εμφάνισε σταθερή συμπεριφορά, με ελαφρώς καλύτερη απόδοση ως προς MSE στο long horizon
- Οι Transformers είχαν αυξημένο υπολογιστικό κόστος και δυσκολία στο long horizon forecasting, υποδηλώνοντας ότι μπορεί να απαιτούν μεγαλύτερα σύνολα δεδομένων

8.2 Σημασία Αποτελεσμάτων

Η απόδοση με MAPE < 3% στο long horizon είναι ιδιαίτερα εντυπωσιακή για εφαρμογές real-world resource prediction. Αυτό σημαίνει ότι τα μοντέλα μπορούν να παρέχουν αξιόπιστες προβλέψεις για τη διαχείριση πόρων σε cloud clusters.

8.3 Προτάσεις για Μελλοντικές Εργασίες

Υπάρχουν αρκετές κατευθύνσεις για περαιτέρω έρευνα:

Chapter 9

Βιβλιογραφία

- [1] <https://arxiv.org/abs/2504.18185>
- [2] <https://www.sciencedirect.com/science/article/pii/S0925231225003844>
- [3] *Natural Hazards*.
<https://link.springer.com/article/10.1007/s11069-025-07408-8>
- [4] https://www.researchgate.net/publication/374260856_Transformers_in_Machine_Learning_Literature_Review
- [5] https://www.tensorflow.org/guide/keras/training_with_built_in_methods
- [6] https://youtu.be/_kp19PIVuXc?si=P-ac59BgloZep-a8
- [7] <https://encord.com/blog/train-val-test-split>
- [8] <https://apxml.com/courses/rnns-and-sequence-modeling/chapter-7-implementing-lstm-gru/using-gru-layers>
- [9] <https://www.lightly.ai/blog/train-test-validation-split>
- [10] https://ml-tutorials.readthedocs.io/en/latest/auto_examples/lstm_tensorflow_scratch.html
- [11] TensorFlow API Documentation. “TransformerEncoder”.
https://www.tensorflow.org/api_docs/python/tfm/nlp/models/TransformerEncoder
- [12] Brownlee, J.
<https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learn>